

6. 사용자 및 기본 보안

미셸 말처¹과 달 쿤²

(1)

헌틀리, 일리노이, 미국

(2)

모리슨, 콜로라도, 미국

데이터베이스에 대한 보안과 접근성은 중요합니다. 데이터베이스는 관리자, 개발자, 애플리케이션, 사용자의 다양한 작업을 수행하기 위해 사용자가 필요합니다.

데이터베이스를 생성하면 몇 개의 기본 계정이 생성됩니다. **23ai**에서는 계정이 스키마에 불과하기 때문에 로그인할 수 없습니다. 이를 통해 데이터베이스의 보안이 강화되고 기본 계정을 통해 데이터베이스에 접근할 수 있는 노출이 제한됩니다.

애플리케이션과 사용자가 데이터베이스에 액세스해야 하므로 새 계정을 생성하고 관리해야 합니다. 여기에는 적절한 인증 방법 선택, 비밀번호 보안 구현, 사용자 권한 할당 등이 포함됩니다.

참고Oracle Database 23ai 이전에는 기본 계정에 비밀번호가 설치되어 있었지만 기본적으로 잠겨 있었습니다. 권장 사항은 계정이 잠겨 있는지 확인하는 것이었습니다. 이제 이러한 계정은 스키마 계정일 뿐 데이터베이스에 로그인할 수 있도록 기본적으로 구성되지 않았습니다.

사용자 유형

오라클 데이터베이스에는 몇 가지 유형의 사용자가 있습니다. 소프트웨어를 설치하고 데이터베이스를 만드는 데 필요한 몇 가지 사용자에게 이미 배웠습니다.

환경의 규모와 환경을 관리할 수 있는 인원 수에 따라 각 역할에 동일한 사람이 있을 수 있습니다. 그러나 대기업의 보안 정책과 업무 분리를 위해 데이터베이스의 보안과 업무를 관리하기 위해 업무와 접근이 서로 다른 역할로 나뉩니다.

다음은 일반적인 오라클 데이터베이스 사용자 목록입니다.

데이터베이스 관리자

보안 담당자

네트워크 관리자

애플리케이션 개발자

애플리케이션 관리자

데이터베이스 사용자

데이터베이스 관리는 일반적으로 오라클 데이터베이스 서버와 도구의 설치 및 업그레이드를 담당하는 데이터베이스 관리자(**DBA**) 그룹에 의해 수행됩니다. **DBA**는 애플리케이션 설계를 기반으로 데이터베이스의 스토리지 및 스토리지 구조를 관리합니다. 백업 및 복구 작업은 객체 생성 및 관리와 함께 **DBA**의 손에 달려 있습니다. 이러한 작업은 시스템 **DBA**의 일반적인 작업이며, **DBA**와 애플리케이션 관리자 또는 애플리케이션 **DBA**의 공동 책임이 있을 수 있습니다.

보안 담당자는 데이터베이스에 사용자와 액세스 권한을 추가합니다. 이들은 데이터베이스에 대한 액세스를 관리 및 모니터링하고 시스템의 보안을 유지하는 역할을 합니다. 보안 정책은 **DBA**가 아닌 보안 팀이 사용자를 생성하도록 구현할 수 있습니다.

데이터베이스의 네트워크 관리자는 데이터베이스 리스너 및 오라클 넷 서비스와 같은 오라클 네트워킹 제품을 관리합니다. 이 관리자는 일반적인 네트워크 관리자는 아니지만 **sqlnet.ora** 설정과 리스너.**ora** 값을 사용하여 전송 중 암호화를 관리하고 리스너 및 **tnsname.ora** 파일에 새 데이터베이스가 추가되는지 확인합니다. **DBA**는 일반적으로 이 작업을 수행하지만 이 역할을 분리할 수 있습니다.

애플리케이션 개발자는 데이터베이스 애플리케이션을 설계하고 구현합니다. 그들은 데이터베이스 객체를 설계하고 개발 중에 애플리케이션을 조정합니다.

데이터베이스에서 작업할 때 **DBA**와 협력하여 애플리케이션에 맞게 데이터베이스 구성이 설계되고 스토리지 및 기타 리소스가 사용 가능한지 확인합니다. **23ai**를 사용하면 데이터베이스 개발자에게 필요한 액세스 권한을 제공하기 위해 **DB_Developer_ROLE**이라는 새로운 역할도 있습니다.

애플리케이션 관리자는 실제로 애플리케이션을 코딩하고 개발하는 것보다 애플리케이션의 설계와 리소스에 더 책임이 있을 수 있습니다. 이 역할은 모든 데이터베이스 리소스를 관리하고, 객체를 생성하고, 성능과 구성을 검증하는 데 도움을 주는 애플리케이션 **DBA**라고도 할 수 있습니다.

데이터베이스 사용자는 애플리케이션이나 데이터베이스 도구를 통해 데이터베이스와 상호 작용합니다. 데이터베이스에 직접 액세스하는 것이 반드시 일반적인 것은 아닙니다. 그러나 데이터베이스 접근 권한과 사용자가 테이블과 데이터를 사용할 수 있는 이유가 있습니다. 데이터베이스 사용자가 사용할 데이터에 대해 데이터를 로드하고 분석 및 쿼리를 수행할 수 있는 도구가 있습니다. 데이터에 접근하는 다른 일반적인 방법으로는 **API**나 데이터베이스 애플리케이션을 사용하는 것이 있습니다.

이러한 사용자 대부분은 데이터베이스에 로그인할 수 있는 데이터베이스 계정을 가지고 있지만 운영 체제 명령을 실행할 필요는 없습니다. 오라클용 **OS** 수준 계정은 데이터베이스 소프트웨어가 설치되고 데이터베이스가 생성될 때 사용되며, 일반적으로 이러한 관리자로 제한됩니다. 서로 다른 운영 체제 계정을 사용할 수 있지만 관리 작업을 수행하려면 데이터베이스 파일과 저장소에 대한 올바른 액세스 권한이 필요합니다. 데이터베이스에 액세스하는 사용자는 운영 체제 액세스가 필요하지 않습니다.

기본 사용자 관리

데이터베이스를 만들 때 **SYS**, **SYSTEM**과 같은 기본 사용자가 있으며,

데이터베이스를 만들 때 비밀번호가 설정됩니다. 비밀번호는 보안 정책으로 변경 및 관리할 수 있습니다.

다음은 관리 사용자 계정 목록입니다.

SYS

시스템

시스템 백업

SYSDG

SYSKM

시스락

관리 계정은 데이터베이스의 다양한 영역을 관리하는 데 필요한 특별 권한을 가지고 있습니다. 이러한 계정에는 또한 **SYS** 스키마에 대한 모든 테이블 또는 변경 세션 권한과 실행 권한이 있습니다. **SYS** 및 **SYS** 계정이 사용되지 않도록 다른 계정을 만들고 일상 업무와 관리자에게 적절한 역할과 권한을 부여하는 것이 좋습니다.

SYSBACKUP은 오라클 복구 관리자(**RMAN**) 백업 및 복구 작업을 위한 것입니다. **SYSDG**는 데이터 가드 브로커 또는 **DGMGRL**을 사용하는 데이터 가드 작업을 위한 것입니다. **SYSKM**은 투명 데이터 암호화를 위한 키 저장소 작업을 수행합니다. **SYSRAC**는 오라클 실제 애플리케이션 클러스터는 **SRVCTL**을 사용하여 데이터베이스를 클러스터웨어에 연결합니다. **SYSRAC**는 데이터베이스 사용자에게 부여할 수 없으며 오라클 에이전트에게만 제공됩니다.

DBA 역할은 데이터베이스와 함께 자동으로 생성됩니다. 이 역할은 데이터베이스 관리자에게만 부여되어야 합니다. **23ai**에서 **DB_DOPEER_ROLE** 역할은 개발자가 애플리케이션 스키마에서 객체를 연결하고 생성할 수 있는 권한을 부여하기 위해 데이터베이스와 함께 생성되는 새로운 역할입니다.

DB_Developer_ROLE은 애플리케이션 객체 생성, **JavaScript** 및 **PL/SQL** 개발 성능 검증, 애플리케이션 내 데이터 수정 등 약 **24**개의 권한을 부여받았습니다. 이 역할을 통해 개발자는 도메인, 작업 및 마이닝 모델을 생성하고, 리소스 역할을 통해 테이블, 뷰, 그래프와 같은 다른 객체(**JSON Duality Views** 등)를 생성할 수 있습니다. **MLE**를 생성하면 개발자는 **JavaScript**용 다국어 엔진을 사용할 수 있습니다.

읽기 전용 사용자

23ai를 사용하면 사용자를 읽기 전용으로 만들 수 있으며, 스키마 승인과 결합하면 해당 스키마의 모든 객체를 읽을 수 있습니다.

SQL> "Passw0rd123!"에 의해 식별된 사용자 **MMRADONLY**를 생성합니다
읽기 전용;

기존 사용자를 읽기 전용으로 변경할 수도 있으므로 해당 사용자에게 부여된 기존 권한은 재정의됩니다. 사용자를 읽기 전용으로 설정한 후 **DBA**를 추가한 후에도 재정의됩니다. 사용자가 다른 권한을 사용하기 전에 쓰기를 다시 읽기 위해 사용자를 변경해야 합니다.

SQL> 사용자 **hsolo** 읽기만 변경합니다;

SQL> 사용자 **hsolo** 읽기 쓰기를 변경합니다;

사용자가 쓰기를 읽기 위해 다시 변경할 수 있기를 기대했다면, **DBA** 권한이 재정의되었으므로 다른 권한 있는 사용자가 이를 다시 변경해야 한다는 점을 기억하세요.

기본적으로 사용자는 **READ WRITE**이지만 사용자가 **READ ONLY**인지 확인하려면 **DBA_USERS**에 문의할 수 있습니다.

SQL> 사용자 이름 선택, **dba_users**에서 **_only** 읽기

사용자 이름='MMALCHER';

사용자 이름 읽기_전용

MMACHER NO

HSOLO YES

스키마 전용 기본 계정

데이터베이스가 생성되면 미리 정의된 여러 스키마 계정이 자동으로 생성됩니다.

23ai의 이러한 계정 대부분은 이제 샘플 계정을 제외하고 스키마 전용 계정으로 정의됩니다. 계정은 설치 중에 잠겨 있고 만료되므로 스키마 전용 계정으로 유지하는 것이 좋습니다. 스키마 전용 계정은 데이터베이스에 로그인할 수 없습니다. 이 계정은 객체를 가질 수 있으며 테이블 및 프로시저와 같은 객체를 생성할 수 있는 시스템 권한을 부여받을 수 있습니다. 이러한 스키마는 프록시 인증에서 클라이언트 사용자를 위해 구성할 수 있습니다. 계정이 스키마 전용인지 확인하려면

DBA_USER 보기를 쿼리하고, 계정이 스키마 전용인지 여부는 **AUTHENTYPE**에 없음으로 표시됩니다.

시스템 대 시스템

오라클 초보자들은 때때로 "**SYS** 스키마와 **SYS** 스키마의 차이점은

무엇인가요?"라고 묻습니다. **SYS** 스키마는 데이터베이스의 슈퍼유저이며, 모든 내부 데이터 사전 객체를 소유하고 있으며 데이터베이스 생성, 인스턴스 시작 또는 중지, 백업 및 복구, 데이터 파일 추가 또는 이동과 같은 작업에 사용됩니다. 이러한 유형의 작업에는 일반적으로 **SYSDBA** 또는 **SYSOPER** 역할이 필요합니다.

이러한 역할의 보안은 종종 오라클 소프트웨어의 **OS** 계정 소유자에 대한 액세스를 통해 제어됩니다.

데이터베이스 및 운영 체제 인증과 글로벌 사용자의 예는 다음 두 섹션에서 설명합니다. 보다 정교한 인증 요구 사항이 있는 경우 네트워크, 글로벌 또는 외부 서비스 인증을 조사해야 합니다. 이러한 방법에 대한 자세한 내용은 오라클 데이터베이스 문서의 일부로 제공되는 오라클 데이터베이스 보안 가이드를 참조하세요.

데이터베이스 인증을 사용한 사용자 생성

데이터베이스 인증은 **CREATE User SQL** 문으로 설정됩니다. **DBA**로 사용자를 만들려면 계정에 **CREATE User** 시스템 권한이 있어야 합니다. 이 예제에서는 비밀번호 **W3lcomHere123**을 사용하여 **HSOLO**라는 사용자를 생성하고 기본 영구 테이블스페이스 사용자, 기본 임시 테이블스페이스 **TEMP** 및 무제한 공간 할당량을 사용자 테이블스페이스에 할당합니다.

SQL> W3lcomHere123에 의해 식별된 사용자 **hsolo** 생성

기본 테이블스페이스 사용자

임시 테이블스페이스 온도

사용자 무제한 할당량;

이렇게 하면 데이터베이스에서 아무것도 할 수 있는 권한이 없는 베어본 스키마가 생성됩니다. 사용자를 유용하게 만들려면 최소한 **CREATE SESSION** 시스템 권한을 부여해야 합니다.

SQL> hsolo에 생성 세션을 부여합니다;

새 스키마가 테이블을 생성할 수 있어야 하는 경우 **CREATE TABLE**과 같은 추가 권한을 부여해야 합니다.

SQL> hsolo에게 테이블 생성 권한 부여;

DB_DOPEER_ROLE 역할이나 테이블 및 스키마에 대한 권한과 같은 역할을 부여할 수 있습니다.

OS 인증을 사용한 사용자 생성

OS 인증은 사용자가 **OS** 사용자 계정에 로그인할 수 있는 경우 데이터베이스 권한을 해당 **OS**와 연결하고 파생할 수 있다고 가정합니다. **OS** 인증에는 두 가지 유형이 있습니다.

사용자에게 특정 **OS** 역할을 할당하여 인증(데이터베이스 권한을 사용자에게 매핑할 수 있음)

외부에서 식별된 조항을 통한 일반 데이터베이스 사용자 인증

OS 역할을 통한 인증은 2장에서 설명합니다. **DBA**는 이러한 유형의 인증을 사용하여 오라클과 같은 **OS** 계정에 연결한 다음 사용자 이름과 비밀번호를 지정하지 않고도 **SYSDBA** 권한으로 데이터베이스에 연결할 수 있습니다.

데이터베이스 서버에 로그인한 후, 외부에서 식별된 절로 생성된 사용자는 사용자 이름과 비밀번호를 지정하지 않고도 데이터베이스에 연결할 수 있습니다. 서버에 접근할 수 있는 사용자는 데이터베이스 사용자 이름과 비밀번호를 유지할 필요가 없습니다.

데이터베이스에 로그인하는 스크립트는 **OS** 인증 사용자가 실행할 경우 하드코딩된 비밀번호를 사용할 필요가 없습니다.

다른 데이터베이스 사용자는 사용자 이름과 비밀번호 연결 문자열을 추측하여 사용자를 해킹할 수 없습니다. **OS** 인증 사용자에게 로그인하는 유일한 방법은 **OS** 에서 로그인하는 것입니다.

OS 인증을 사용할 때 오라클은 **OS_AUTHENT_PREFIX** 데이터베이스 초기화 매개변수에 포함된 값을 데이터베이스에 연결하는 **OS** 사용자에게 접두사로 붙입니다. 이 매개변수의 기본값은 **OPS\$**이지만 이 값을 널 문자열로 설정하는 것이 좋습니다.

SQL> 시스템 세트 **os_authent_prefix = " scope=spfile** 변경
이 수정 사항을 적용하려면 데이터베이스를 중지하고 시작해야 하므로 데이터베이스를 만들 때 **OS** 인증 및 매개변수 설정을 사용하기로 결정해야 합니다. **OS** 인증을 받은 사용자가 데이터베이스에 액세스할 수 있도록 하려면 여전히 사용자를 만들어야 합니다.

SQL> 외부에서 식별된 사용자 **jsmith**를 만듭니다;

SQL> 생성 세션을 **jsmith**에게 부여합니다;

이제 **jsmith**가 데이터베이스 서버에 로그인하면, 이 사용자는 다음과 같이

SQL *Plus에 연결할 수 있습니다.

\$ sqlplus /

OS가 이미 사용자를 인증했기 때문에 사용자 이름이나 비밀번호가 필요하지 않습니다.

중앙 관리 사용자 구성

중앙에서 관리되는 사용자는 **Active Directory** 또는 다른 **LDAP** 서비스와 같은 곳의 사용자로 간주됩니다. 사용자는 중앙에서 인증 및 권한을 관리할 수 있으며, **Active Directory**의 사용자는 비밀번호나 다른 유형의 키를 통해 인증을 관리하고 보안 그룹을 사용하여 권한을 관리할 수 있습니다. 사용자가 보안 그룹을 변경하면 권한이 변경되며, **Active Directory**에서 비활성 상태이면 다른 애플리케이션이나 데이터베이스에 인증할 수 없습니다.

데이터베이스에서 사용자를 글로벌 사용자로 만들 수 있습니다. 즉, 데이터베이스는 **Active Directory**에 연락하여 사용자에 대한 세부 정보를 얻고, 먼저 비밀번호를 인증한 다음 그룹을 인증합니다. 데이터베이스는 **Active Directory**에 대한 사용자로 구성되며, **Idap.ora** 파일은 **Active Directory**에 대한 인증 정보와 함께 업데이트됩니다. 일단 구성되면 다음 구문을 사용하여 사용자를 만들 수 있습니다.

SQL>은 'cn=hsolodba group,ou=dbat eam,dc=example,dc=com'으로 전역적으로 식별되는 사용자 **hsolodba**를 생성합니다;

이를 통해 오라클 데이터베이스는 **Active Directory**에 있는 사용자 **hsolodba**를 이 데이터베이스에 액세스할 수 있는 사용자로 인식할 수 있습니다. 비밀번호는 **Active Directory**에 있는 비밀번호와 동일하며, 이 그룹은 권한을 위해 데이터베이스 역할에 매핑하는 데 사용할 수 있습니다.

신원 관리는 기업에서 중요하며, 개인이 기업 내에서 자신의 기능에 따라 역할과 작업을 수행할 수 있도록 합니다. 이러한 기능은 변경되거나 더 이상 회사와 함께하지 않으므로 계정은 각 데이터베이스가 아닌 중앙에서 관리됩니다. 오라클 클라우드에는 역할을 구성하고 사용자를 추가할 수 있는 신원 접근 관리(**OCI IAM**)가 있습니다. 이러한 관리는 각 데이터베이스가 아닌 신원 관리 서비스의 일부로 관리됩니다.

사용자를 **OCI IAM**으로 가져올 수 있으므로 각 계정을 입력할 필요가 없습니다. 모든 엔터프라이즈 사용자를 가져올 수는 없지만 이러한 방식으로 작업하면 어떤 사용자를 마이그레이션해야 하는지 고려한 다음 역할을 확인할 수 있습니다.

일반 및 로컬 사용자

각 **CDB**와 **PDB**에는 유효한 데이터베이스 사용자 목록이 있습니다. 일반 사용자는 다양한 컨테이너에 액세스할 수 있으며 **CDB**에 속해 있습니다. 로컬 사용자는 **PDB**에 특화되어 있습니다.

SYS 및 **SYSTEM** 계정은 오라클이 플러그 가능한 환경에서 자동으로 생성하는 일반 사용자이며 시스템 컨테이너를 탐색할 수 있습니다. **CDB** 일반 사용자는 서로 다른 **PDB**에서 서로 다른 권한을 가질 수 있습니다.

공용 사용자는 사용자 이름의 시작 부분에 **C##** 또는 **c##**으로 생성됩니다.

COMMON_USER_PREFIX 매개변수는 접두사를 설정하며, 접두사를 변경할 수 있더라도 이름 충돌이 발생할 수 있으므로 신중하게 처리해야 합니다. 이 예제는 **CDB**에서 모든 **PDB**에 공용 사용자를 생성합니다.

SQL> 사용자 **c##dba**를 "**W3lcome1234**"로 식별합니다;

일반 사용자는 각 플러그 가능한 데이터베이스 내에서 권한을 부여받아야 합니다. 즉, 루트 컨테이너에 연결된 상태에서 공통 사용자 권한을 부여하는 것은 **PDB**에 연쇄적으로 적용되지 않습니다. 일반 사용자에게 **PDB**를 포괄하는 권한을 부여해야 하는 경우 공통 역할을 생성하여 일반 사용자에게 할당합니다.

일반 사용자에게 어떤 용도가 있나요? 한 가지 상황은 **SYSDBA** 수준의 권한이 필요하지 않은 **PDB** 전반에 걸친 일반적인 **DBA** 유지 관리 활동의 수행입니다. 예를 들어, 사용자, 보조금 등을 생성할 수 있는 권한을 가진 **DBA** 계정을 설정하고 싶지만 모든 데이터베이스에 모든 권한을 가진 **SYS**와 같은 계정을 사용하지는 않습니다. 이 시나리오에서는 공통 **DBA** 사용자와 적절한 권한을 포함하는 **DBA**

공통 역할을 생성하게 됩니다. 그런 다음 공통 역할이 공통 **DBA**에 할당됩니다.

로컬 사용자는 **PDB**에서 생성되는 일반 사용자입니다. 로컬 사용자는 이전 섹션에서와 같이 생성됩니다. 로컬 사용자 이름은 생성된 **PDB**에만 고유해야 합니다. 로컬 사용자는 로컬 계정이 생성된 **PDB**에 대해서만 관리 권한을 가질 수 있습니다. 역할도 공통 또는 로컬 역할입니다. 오라클에서 제공하는 모든 역할은 공통이지만 로컬 사용자에게 부여할 수 있습니다.

스키마 대 사용자 이해

스키마는 데이터베이스 객체(여기서 테이블 및 인덱스)의 모음입니다. 사용자는 사용자 이름과 비밀번호를 사용하여 데이터베이스에 연결하는 계정입니다. 사용자는 객체를 소유할 수도 있으며, 객체의 스키마는 소유자가 됩니다. 데이터베이스 시스템 객체를 소유하기 위해 생성된 기본 스키마에 대해 이미 배웠습니다.

사용자로 연결할 때는 기본적으로 데이터베이스에 연결하는 사용자가 소유한 스키마에서 객체를 조작할 수 있습니다. 예를 들어, 테이블을 설명하려고 할 때 오라클은 기본적으로 현재 사용자의 스키마에 액세스합니다. 따라서 현재 연결된 사용자(소유자)에게 테이블 이름을 미리 입력할 이유가 없습니다. 현재 연결된 사용자가 **INV_MGMT**라고 가정해 보겠습니다. 다음 설명 명령을 고려해 보세요.

SQL> 인벤토리 설명;

이전 진술은 다음 진술과 기능이 동일합니다.

SQL> `desck inv_mgmt.inventory;`

현재 사용자의 세션을 **ALTER SESSION** 문을 통해 다른 스키마를 가리키도록 변경할 수 있습니다.

SQL> 세션 세트 `current_schema = 판매` 변경;

이 문장은 현재 사용자(이 예시에서는 **INV_MGMT**)에게 추가 권한을 부여하지 않습니다.

이 문장은 오라클에게 데이터베이스 객체를 참조하는 후속 **SQL** 문에 대해 스키마 한정자 **SALES**를 사용하도록 지시합니다. 적절한 권한이 부여된 경우

INV_MGMT 사용자는 스키마 이름을 객체 이름 앞에 붙이지 않고도 **SALES**

사용자의 객체에 액세스할 수 있습니다.

설명과 설명이 동일한 함수인 것처럼, 주문 테이블을 설명하는 것은 **SALES.ORDRS**를 사용하는 것과 동일합니다.

SQL> 판매주문 설명

변경 세션이 **SALES**로 설정되면 결과는 동일합니다.

SQL> 설명 주문

스키마 전용 계정

생성된 일부 기본 계정과 마찬가지로 스키마 전용 계정도 만들 수 있습니다. 이 계정은 객체를 소유하고 데이터베이스에 로그인하는 계정으로 사용되지 않도록 설계되었으며 비밀번호 없이 생성됩니다. 이는 애플리케이션 스키마에 이상적이며 모든 객체를 보유하므로 이러한 계정에 액세스할 수 있는 권한이 부여되면 이러한 객체에 대한 변경을 수행할 수 있습니다. 이러한 계정은 데이터베이스에 직접 로그인할 수 있는 권한이 없습니다. 따라서 비밀번호가 없어도 로그인할 가능성이 없으며 오류가 발생합니다.

SQL> 사용자 앱 생성1 인증 없음;

dba_users 테이블에서 이 사용자는 **AUTHENTYPE=none**를 가지고 있으며, 비밀번호 열은 **NULL**입니다. 스키마 전용 계정은 테이블 및 기타 객체를 생성할 수 있는 권한을 부여받을 수 있지만, 관리 권한을 부여받을 수는 없습니다. 스키마 전용 계정에 **CREATE SESSION**을 부여하더라도 이 스키마 계정에 직접 로그인할 수는 없습니다.

SQL> 앱1에 생성 세션을 부여합니다;

그랜트가 성공했습니다.

SQL> 연결 앱1

비밀번호 입력:

오류

ORA-01005: null 비밀번호가 지정되었으며, 로그인이 거부되었습니다

경고 더 이상 오라클에 연결되어 있지 않습니다.

프록시 연결을 통해 스키마 전용 계정을 포함한 다른 계정에서 **DDL** 문을 수행할 수 있습니다. 다음은 데이터베이스에 로그인할 때 **hsolosdba**를 계정으로 사용하는

예시입니다. 스키마 전용 계정은 **app1** 입니다

SQL> hsolodba를 통해 사용자 앱1 승인 연결을 변경합니다;

SQL> 연결 hsolodba[app1]/Pa33word 안녕하세요!

SQL>에서 sys_context('USERENV', 'SESSION_USER')를 세션_사용자로 선택합니다,

sys_context('USERENV', 'SESSION_SCHEMA'): 세션 스키마, **sys_context('USERENV', 'PROXY_USER')**를 **proxy_id**, 사용자로 사용합니다;
세션_사용자 세션_Schema 프록시_ID 사용자

APP1 앱 HSOLODBA 앱

맞습니다. **23ai**에서는 **FROM** 절이 필요하지 않습니다. 이전 버전에서는 쿼리에서 **FROM DUAL**을 사용해야 했습니다. **23ai**에서는 스키마에 권한을 부여할 수 있으며, 각 객체에 대한 권한일 필요는 없습니다.

SQL> grant 스키마 **APP_SCHEMA**에서 임의의 테이블을 선택합니다;
이 스키마는 비밀번호 없이 데이터베이스의 객체에 대한 애플리케이션 스키마로 간단히 사용할 수 있도록 생성할 수 있습니다. 해당 스키마의 모든 객체에 대해 스키마에 권한을 부여할 수 있습니다. 이 스키마 계정으로 로그인할 수 없으며, 스키마 전용 계정은 객체의 소유자 또는 코드 및 절차를 실행할 소유자일 뿐입니다. 스키마 전용 계정은 객체의 소유자 또는 코드와 프로시저를 실행할 소유자일 뿐입니다. 이는 응용 스키마에 고려해야 할 사항이며, 다가오는 "권한 관리" 섹션에서 설명한 바와 같이 이러한 객체에 대한 권한과 권한은 여전히 역할별로 처리할 수 있습니다.

기본 영구 및 임시 테이블스페이스 할당

사용자가 올바른 기본 영구 테이블스페이스와 임시 테이블스페이스를 가지고 있는지 확인하면 의도치 않게 **SYSTEM** 또는 **SYS_AUX** 테이블스페이스를 채우는 것을 방지할 수 있으며, 이로 인해 데이터베이스를 사용할 수 없게 되어 성능 문제가 발생할 수 있습니다. **USERS** 테이블스페이스는 일반적으로 기본 테이블스페이스입니다.

데이터베이스를 유지 관리할 때는 기본 영구 및 임시 테이블스페이스 설정을 확인하여 데이터베이스 표준을 충족하는지 확인해야 합니다. 사용자 정보는 **DBA_USER** 보기에서 선택하여 확인할 수 있습니다.

SQL> dba_users에서 사용자 이름, 비밀번호, 기본값 **tablespace**,
임시 **tablespace**를 선택합니다;

다음은 출력의 작은 샘플입니다.

사용자 이름 비밀번호 기본_테이블스페이스 임시_테이블스페이스

JSMITH 외부 사용자 **TEMP**

MDDATA 사용자 온도

시스템 백업 사용자 임시 근무 시간

모든 사용자에게는 임시 형식으로 생성된 임시 테이블스페이스가 할당되어야 합니다.

일반적으로 이 테이블스페이스는 **TEMP**라고 하며, **PDB** 또는 **CDB**에만 임시
테이블스페이스가 있을 수 있습니다.

부적절한 기본 테이블스페이스 설정을 가진 사용자를 찾거나 다른 임시
테이블스페이스로 변경하려는 경우 **ALTER USER** 문으로 수정할 수 있습니다.

SQL>은 사용자 **inv_mgmt** 기본 테이블스페이스 사용자 임시 테이블스페이스
temp2를 변경합니다;

SYSTEM은 임시 테이블스페이스로 할당되어서는 안 되며, **SYSTEM**과
SYSAUX는 사용자에게 할당되어서는 안 됩니다.

사용자 수정

때때로 다음과 같은 이유로 기존 사용자를 수정해야 할 때가 있습니다.

사용자 비밀번호 변경

사용자 잠금 또는 잠금 해제

기본 영구 또는 임시 테이블스페이스 변경

프로필 또는 역할 변경

시스템 또는 객체 권한 변경

테이블스페이스의 할당량 수정

ALTER User 문을 사용하여 사용자를 수정합니다. 다음은 사용자를 수정하는 여러 **SQL** 문입니다. 이 예제에서는 **IDENTED BY** 절을 사용하여 사용자의 비밀번호를 변경합니다.

SQL>은 **W3lcomHere123**에 의해 식별된 사용자 **inv_mgmt**를 변경합니다;
이 예제는 사용자 계정을 잠급니다.

SQL> 사용자 **inv_mgmt** 계정 잠금 변경;

이 예제는 사용자 테이블스페이스에서 사용자의 할당량을 변경합니다.

SQL> 사용자 **inv_mgmt** 할당량 **500G**를 사용자에게 변경합니다;

참고 **ALTER** 사용자는 매우 특권적인 명령어이며 이를 사용하는 이유가 많기 때문에 이제 보안 팀의 손에 넘어가 실행할 수 있습니다. 이와 관련하여 다른 명령어와 절차를 작성한 다음 실행할 사람에게 권한을 부여할 수 있습니다. 또한 오라클 보안 옵션인 오라클 데이터베이스 볼트는 사용자를 변경할 수 있는 기능을 제한하고 보안 팀이 이러한 작업을 수행할 수 있도록 허용합니다.

사용자 삭제

사용자를 삭제하기 전에 먼저 사용자를 잠그는 것이 좋습니다. 사용자를 잠그면 다른 사용자가 잠긴 데이터베이스 계정에 연결할 수 없습니다. 이렇게 하면 계정이 삭제되기 전에 다른 사용자가 계정을 사용하고 있는지 여부를 더 잘 확인할 수 있습니다. 다음은 사용자를 잠그는 예입니다.

SQL> 사용자 **hsolodba** 계정 잠금 변경;

이 사용자에게 연결을 시도하는 모든 사용자 또는 애플리케이션은 이제 다음과 같은 오류를 받습니다.

ORA-28000: 계정이 잠겨 있습니다

이 쿼리를 발행하여 데이터베이스에서 사용자와 잠금 날짜를 확인하세요.

SQL> 사용자 이름을 선택하고 **dba_users**에서 **lock_date**를 선택합니다;
계정을 잠금 해제하려면 이 명령을 내립니다.

SQL> 사용자 **hsolodba** 계정 잠금 해제 변경;
사용자 잠금은 데이터베이스를 보호하고 활성화된 사용자를 식별하는 데 유용한
기술입니다.

사용자를 잠그면 사용자의 객체에 대한 액세스가 잠기지 않는다는 점을 유의하세요.
예를 들어, 사용자 **A**가 사용자 **B**가 소유한 테이블에 대한 선택, 삽입 업데이트 및
삭제 권한을 가지고 있다면, 사용자 **B** 계정을 잠그면 사용자 **A**는 여전히 사용자 **B**
가 소유한 객체에 대해 **DML** 문을 발급할 수 있습니다. 객체가 사용되고 있는지
확인하기 위해 객체가 감사됩니다. (객체 감사에 대한 자세한 내용은 **20**장에서
확인할 수 있습니다.)

사용자에게 유효한 객체 백업이 있는지 또는 빠른 백업을 수행하는지 확인할 가치가
있습니다. 데이터베이스에 여전히 객체를 소유하고 있는 사용자는 삭제할 수
없습니다. 그렇기 때문에 애플리케이션 객체는 개별 계정으로 모든 객체를 생성하는
대신 다른 스키마에 넣어야 합니다. 애플리케이션을 폐기하는 경우 백업 및 보존
정책도 고려해야 합니다.

사용자와 그 객체가 필요하지 않다는 것을 확인한 후, **DOP User** 문을 사용하여
데이터베이스 계정을 제거합니다. 이 예제에서는 사용자 **hsolodba**를 삭제합니다.

SQL> 드롭 사용자 **hsolodba**;

사용자가 데이터베이스 객체를 소유하고 있으면 이전 명령이 작동하지 않습니다.

CASCADE 절을 사용하여 사용자를 제거하고 해당 객체를 삭제합니다.

SQL> 드롭 사용자 **hsolodba** 캐스케이드;

드롭되는 사용자가 방대한 수의 데이터베이스 객체를 소유하고 있는 경우, 드롭
사용자 문을 실행하는 데 많은 시간이 걸릴 수 있습니다. 이러한 상황에서는 사용자를
떨어뜨리기 전에 사용자의 물건을 떨어뜨리는 것을 고려할 수 있습니다.

프로필

사용자를 만들 때 강력한 비밀번호와 일련의 보안 규칙을 준수해야 한다는 요구 사항이 있습니다. 특정 사용자가 과도한 양의 **CPU** 리소스를 소비할 수 없도록 해야 할 수도 있습니다. 이 두 가지 예는 데이터베이스 프로필에서 수행할 수 있습니다. 오라클 프로필은 두 가지 목적을 수행하는 데이터베이스 개체입니다. 비밀번호 보안 설정을 적용합니다

사용자가 소비하는 시스템 리소스를 제한합니다

팁 데이터베이스 프로필과 **SQL** 프로필을 혼동하지 마세요. 데이터베이스 프로필은 비밀번호 보안을 강제하고 데이터베이스 리소스 사용을 제한하는 사용자에게 할당된 개체입니다. 반면 **SQL** 프로필은 **SQL** 문과 연관되어 있으며 최적화 도구가 보다 효율적인 실행 계획을 생성하는 데 도움이 되는 통계 수정 사항을 포함합니다.

데이터베이스 리소스 사용 제한

앞서 언급했듯이 비밀번호 프로필 설정은 사용자에게 프로필을 할당할 때 적용됩니다. 비밀번호 설정과 달리 커널 리소스 프로필 제한은 데이터베이스의 기본값인 **RESOURCE_LIMIT** 초기화 매개변수를 **TRUE**로 설정할 때까지 적용되지 않습니다. 다음은 예입니다.

SQL> 시스템 세트 **resource_limit=true scope=both**를 변경합니다;
Resources_LIMIT 매개변수의 현재 설정을 보려면 이 쿼리를 발행하세요.

SQL > 이름을 선택하고 **v\$parameter**에서 값을 선택합니다. 여기서
name='resource_limit';

사용자를 생성할 때 프로필을 지정하지 않으면 사용자에게 기본 프로필이 할당됩니다. **ALTER PROFILE** 문을 사용하여 기본 프로필을 수정할 수 있습니다. 다음 예제에서는 **CPU_PER_SESSION**을 240,000(수백 초 만에)으로 제한하도록 기본 프로필을 수정합니다.

SQL> 프로필 기본 제한 **cpu_per_session 2400**을 변경합니다;
이렇게 하면 기본 프로필을 사용하는 모든 사용자가 **CPU** 사용 시간을 **2,400**초로 제한할 수 있습니다. 프로필에서 다양한 제한을 설정할 수 있습니다. 다음은 제한할 수 있는 데이터베이스 리소스 설정의 예입니다.

CONNECT_TIME

CPU_PER_SESSION

유틸_시간

논리_읽기_per_SESSION

세션_PER_USER

사용자 지정 프로필을 생성하여 **CREATE PROFILE** 문을 통해 사용자에게 할당할 수도 있습니다. 그런 다음 해당 프로필을 기존 데이터베이스 사용자에게 할당할 수 있습니다. 다음 **SQL** 문은 **CPU** 및 개별 세션이 소비할 수 있는 양과 같은 리소스를 제한하는 프로필을 생성합니다.

프로필 생성하기 **user_profile_limit**

제한

세션_per_user 20

cpu_per_session 2400

논리_reads_per_session 1000000

connect_time 480

유틸_시간120;

프로필을 만든 후에는 사용자에게 프로필을 할당할 수 있습니다. 다음 예제에서는 사용자 **hsolo**에게 **USER_PROFILE_LIMIT**이 할당됩니다.

SQL> 사용자 **hsolo** 프로필 **user_profile_limit**을 변경합니다;

권한 관리

데이터베이스 사용자는 데이터베이스에서 작업을 수행하기 전에 권한을 부여받아야 합니다. **Oracle**에서는 사용자에게 특정 권한을 부여하거나 역할에 권한을 부여한 다음 권한이 포함된 역할을 부여하여 권한을 할당합니다. 권한에는 시스템 권한, 스키마 권한, 객체 권한 등 다양한 유형이 있습니다.

데이터베이스 시스템 권한 할당

데이터베이스 시스템 권한을 사용하면 데이터베이스에 연결하여 개체를 생성하고 수정할 수 있습니다. 수백 가지의 다양한 시스템 권한이 있습니다.

DBA_SYS_PIRS 보기를 쿼리하여 시스템 권한을 볼 수 있습니다.

SQL> dba_sys_privs에서 고유 권한을 선택합니다;

시스템 권한은 **CDB**와 **PDB**에서 동일합니다. 다른 사용자나 역할에 권한을 부여할 수 있습니다. 권한을 부여하려면 사용자에게 모든 권한 부여 권한이 필요하거나

ADMIN 옵션이 있는 시스템 권한이 부여되어야 합니다.

GRANT 문을 사용하여 사용자에게 시스템 권한을 할당합니다. 예를 들어, 최소한 사용자가 데이터베이스에 연결하려면 **CREATE SESSION**이 필요합니다. 다음과 같이 이 시스템 권한을 부여합니다.

SQL> 생성 세션을 **inv_mgmt**에 부여합니다;

일반적으로 사용자는 데이터베이스에 연결하는 것 이상의 작업을 수행해야 합니다.

예를 들어, 사용자는 테이블 및 기타 데이터베이스 개체를 만들어야 할 수도 있습니다. 이 예제에서는 사용자에게 테이블 생성 및 데이터베이스 링크 생성 시스템 권한을 부여합니다.

SQL> 생성 테이블을 부여하고 **inv_mgmt**에 대한 데이터베이스 링크를 생성합니다;

스키마 전용 계정도 마찬가지로입니다.

SQL> 테이블 생성 권한 부여, **app1**에 대한 데이터베이스 링크 생성;

모든 개별 권한을 사용하면 역할을 더 쉽게 사용할 수 있습니다. **DBA** 및 **DB_DOPEER_ROLE**과 같은 역할이 제공되는지 여부와 상관없이 잠시 동안 역할에 대한 자세한 내용을 직접 만들 수 있습니다.

SQL> hsolo_dev에게 **db_developer_**역할을 부여합니다;

권한을 제거해야 하는 경우 **RECOBE** 문을 사용합니다.

SQL> inv_mgmt에서 테이블 생성 취소;

오라클에는 사용자에게 시스템 권한을 부여하고 해당 사용자에게 권한을 관리할 수 있는 기능이 있습니다. 이 작업은 **WITH ADMIN Option** 조항을 사용하여

수행합니다.

SQL> 관리 옵션을 사용하여 **inv_mgmt**에 생성 테이블을 부여합니다;
WITH ADMIN Option을 부여하면 권한 관리 및 모니터링을 쉽게 할 수 있습니다.
프로덕션 환경에서는 이러한 사용을 제한하고 역할 및 권한 부여에 대한 보안 제어 기능을 갖추는 것이 좋습니다.

데이터베이스 객체 권한 할당

데이터베이스 객체 권한을 사용하면 다른 사용자의 객체에 액세스하고 조작할 수 있습니다. 권한을 부여할 수 있는 데이터베이스 객체의 유형에는 테이블, 보기, 구체화된 보기, 시퀀스, 패키지, 기능, 절차, 사용자 정의 유형, 디렉토리 등이 있으며, 이제 **23ai** 스키마가 적용됩니다. 객체 권한을 부여하려면 다음 중 하나가 참이어야 합니다
당신은 그 물건을 소유하고 있습니다.

객체에 **GRANT OPTION**이 부여되었습니다.

귀하는 모든 객체 권한 부여 시스템 권한을 가지고 있습니다.

이 예제는 **INV_MGMT_APP** 사용자에게 객체 소유자로서 객체 권한을 부여합니다(객체 소유자로서).

SQL> 등록 시 **inv_mgmt_app**에 삽입, 업데이트, 삭제 선택을 허용합니다;
GRANT ALL 문은 객체에 **INSERT, UPDATE, DELETE, SELECT**를 부여하는 것과 동일합니다. **ALL** 명령에는 **ALTER, INDEX, REFERENES, READ, ON Commit REFRESH, QUER REWITE, DEBUG, FLASHBACK**과 같은 다른 권한이 포함됩니다. 다음 문장은 이전 문장과 동일합니다.

SQL> 등록 시 모든 것을 **inv_mgmt_app**에 부여합니다;
열 수준의 테이블에 **INSERT** 및 **UPDATE** 권한을 부여할 수도 있습니다. 다음 예제에서는 인벤토리 테이블의 특정 열에 **INSERT** 권한을 부여합니다.
SQL> 인벤토리에 삽입된 삽입물(**inv_id, inv_name, inv_dec**)을

inv_mgmt_app에 부여합니다;

객체 권한을 제거해야 하는 경우 **REVOKE** 문을 사용합니다. 이 예제에서는 **INV_MGMT_APP** 사용자로부터 **DML** 권한을 취소합니다.

SQL> inv_mgmt_app에서 삽입 취소, 업데이트, 삭제 등록 시 선택;

그룹화 및 권한 할당

역할은 시스템 또는 객체 권한을 논리적으로 그룹화하거나 둘 다 사용할 수 있게 해주는 데이터베이스 객체로, 이러한 권한을 한 번에 사용자에게 할당할 수 있습니다.

다른 역할에도 역할을 부여할 수 있지만, 너무 많은 역할을 중첩하면 관리하기 어렵기 때문에 주의해야 합니다. 역할은 중앙 개체에 할당된 권한을 제공하여 데이터베이스 보안 측면을 관리하는 데 도움이 됩니다. 이후 여러 사용자 또는 다른 역할에 역할을 할당할 수 있습니다.

역할을 만들려면 **CREATE ROLE** 시스템 권한을 가진 사용자로 데이터베이스에 연결합니다. 다음으로 역할을 생성하고 그룹화하려는 시스템 또는 개체 권한을 할당합니다. 이 예제에서는 **CREATE ROLE** 문을 사용하여 **JR_DBA** 역할을 만듭니다.

SQL> 역할 jr_dba 생성

다음 몇 줄의 **SQL**은 새로 생성된 역할에 시스템 권한을 부여합니다.

SQL> grant는 **jr_dba**에 원하는 테이블을 선택합니다;

SQL> grant는 **jr_dba**에 임의의 테이블을 만듭니다;

SQL> grant는 **jr_dba**에 대한 모든 보기를 생성합니다;

SOL> grant는 **jr_dba**의 동의어를 생성합니다;

SOL> 생성 세션을 jr_dba에 부여합니다;

SOL> grant 데이터베이스 생성 링크 **jr_dba**;

다음으로, 해당 권한을 갖고 싶은 모든 사용자에게 역할을 부여하세요.

SOL> jsmith에게 **jr_dba**를 부여합니다;

SOL> 워커에게 **jr_dba**를 부여합니다;

이제 **JSMITH**와 **LWARK** 사용자는 동의어 및 보기 생성과 같은 작업을 수행할 수 있습니다. 역할이 할당된 사용자를 보려면 **DBA_ROLE_PIRS** 보기를

쿼리하세요.

SOL> 부여자를 선택하고, **dba_role_privs** 순서에서 1 만큼 부여된_역할을 선택합니다;

현재 연결된 사용자에게 부여된 역할을 보려면 **USER_ROLE_PIRS** 보기에서 쿼리하세요.

SOL> **user_role_privs**에서 *를 선택합니다;

역할에서 권한을 취소하려면 **REVOCE** 명령을 사용합니다.

SOL> **jr_dba**에서 데이터베이스 링크 생성 취소;

마찬가지로, **REVOCE** 명령을 사용하여 사용자로부터 역할을 제거합니다.

SOL> **lwalker**에서 **jr_dba**를 취소합니다;

다른 데이터베이스 개체와 달리 역할에는 소유자가 없습니다. 역할은 할당된 권한에 따라 정의됩니다.

스키마 권한

23ai를 처음 사용하면 이제 사용자에게 스키마를 부여할 수 있습니다. 이는 스키마의 모든 객체가 부여된다는 것을 의미합니다. 이렇게 하면 새 객체가 추가되거나 수정되면 스키마를 기반으로 한 부여가 간소화됩니다. 새 테이블과 객체는 해당 테이블에 대한 액세스 권한을 특별히 부여하지 않고도 액세스할 수 있습니다.

이렇게 하면 데이터베이스 객체에 대한 보조금 관리가 간소화됩니다. 일반적으로 보조금은 변경하기 전에 저장된 후 객체 자체에 다시 추가되거나 유지됩니다. 스키마 수준에서는 새로운 객체나 변경 사항에 대한 권한이 이미 사용 가능합니다.

역할을 생성하고 권한을 부여하는 것은 여전히 권장되지만, 개별 사용자에게도 권한을 부여할 수 있습니다.

SQL> **grant**는 임의의 테이블을 선택하고, 임의의 테이블을 삽입하고, 임의의 테이블을 업데이트하고, 임의의 테이블을 삭제합니다

스키마 **app1**에서 **hsolo**까지 모든 테이블을 테이블로 표시합니다;

SQL> **grant**는 임의의 테이블을 선택하고 스키마 **app1**에서 **app1**에 임의의 테이블을 삽입합니다._

개발자 역할;

SQL>은 스키마 **app1**에서 **app_user_role**에 대한 모든 절차를 실행합니다; 스키마 수준 권한은 감사 추적에 표시됩니다. 스키마에서 권한을 부여할 수 있는 것처럼 권한도 취소할 수 있습니다.

일부 권한은 스키마에 부여할 수 있는 권한에서 제외되며, 이는 관리 및 시스템 수준의 권한이기 때문에 합리적입니다. 다음은 제외된 권한입니다.

SYSDBA

SYSOPER

시스마스

시스템 백업

SYSDG

SYSKM

생성과 같은 권한을 부여할 수 있습니다.

SQL> grant는 스키마 **app1**에서 **app_dev_role**로 임의의 테이블을 만듭니다; 스키마 수준에서 권한을 부여할 수 있으면 개체가 변경될 때 권한 부여가 간단해집니다. 이러한 권한을 가진 역할을 생성하여 사용자의 권한을 관리하는 것이 여전히 합리적입니다.

PL/SQL 및 역할

PL/SQL로 작업할 경우, 절차나 함수를 컴파일하려고 할 때 때때로 이 오류가 발생할 수 있습니다.

PL/SQL: ORA-00942: 테이블이나 보기가 존재하지 않습니다
혼란스러운 점은 표를 설명할 수 있다는 것입니다.

SQL> desck app_table;

PL/SQL 이 테이블을 인식하지 못하는 이유는 무엇인가요? **PL/SQL**은 패키지, 프로시저 또는 함수의 소유자에게 코드에서 참조되는 모든 객체에 대한 권한을 명시적으로 부여해야 하기 때문입니다. **PL/SQL** 코드 소유자는 역할을 통해 권한을 얻을 수 없습니다.

이 문제가 발생하면 **PL/SQL** 코드의 소유자로 시도해 보세요.

SQL> 역할 없음 설정;

이제 해당 테이블에 액세스하는 **SQL** 문을 실행해 보세요.

SQL> app_table에서 **count(1)**을 선택합니다;

더 이상 테이블에 액세스할 수 없는 경우 역할을 통해 액세스 권한이 부여되었습니다. 이 문제를 해결하려면 테이블 소유자로서 **PL/SQL** 코드 소유자에게 테이블에 대한 액세스 권한을 명시적으로 부여하세요.

SQL> 소유자/패스 연결

SQL> proc_owner에게 **app_table**에서 선택한 승인;

PL/SQL 코드 소유자로서 연결하여 코드를 성공적으로 컴파일할 수 있어야 합니다.

역할은 사용자가 수행할 수 있는 기능이나 작업에 필요한 권한을 부여하는 방법을 제공합니다. 역할은 사용자가 보안 그룹에 매핑할 때 권한을 관리하는 가장 좋은 방법입니다. 다양한 객체와 시스템 권한에 대한 역할 기반 액세스를 통해 간단한 감사를 통해 누가 역할을 가지고 있는지 확인하고 개별 권한이 부여되지 않았는지 확인할 수 있습니다.